DEVIATHON 2.0 · DEV_103 · TEAM SOYACHAAP

OpsLens
the complete guide

Every ticket is a promise.

What we built, every decision and why, and every file explained —
in simple words, for anyone new to the project.

Source: docs/GUIDE.md · every number from eval/results.json

Written 24 Oct 2026 for Jijnasu (the owner) and for anyone new to the project. Read it from top to bottom once. After that, use the table of contents to jump. Simple words on purpose.

Every number in this guide comes from `eval/results.json` (the results file). If a number here and a number on the screen ever disagree, the results file wins.

Table of contents

1. The whole project in two minutes
2. Words you need
3. The problem statement, and how we answered each line
4. The big decisions — what we chose, why, and why not the other option
5. How the engine works, step by step
6. What we can say — the results
7. What failed, and what we learned
8. The dashboard, screen by screen
9. The repository, folder by folder and file by file
10. How to run everything
11. How we worked (two people, two Claudes, one repo)
12. The timeline
13. What is still open
14. If you only remember ten things

1. The whole project in two minutes

The competition. Deviathon 2.0 finale (29–31 Oct 2026, GLA University). Our problem is DEV_103 from Eternal Ltd (the company behind Zomato and Blinkit): "*AI-Powered Business Operations Intelligence & Decision Engine*". Team SoyaChaap: Jijnasu (owner, tech) and Sneha (story, demo).

The problem in one sentence. A company's support team gets about a thousand complaints a day; each complaint has a deadline (an SLA); some kinds of complaints are always late, some teams are overloaded, and urgent ones miss their deadline. Build something that finds *why*, predicts *what will go wrong next*, recommends *what to do*, explains *why*, and measures *how much it helps*.

What we built: OpsLens. A decision engine for a support floor. It:

1. reads each complaint and works out what it is about (also in Hinglish);
2. finds recurring problems in the history, with proof;
3. predicts the next 48 hours and which waiting customers will miss their deadline, with reasons;
4. decides who should take which complaint right now, and recommends bigger fixes;
5. **proves all of this** by replaying days it never saw in a simulator, against the usual ways of working.

The headline. On 14 days OpsLens never saw, with the same customers and the same agents, **2.0%** of customers missed their deadline with OpsLens, against **5.5%** with the industry-standard way of routing work. The difference is −3.46 points, and the 95% interval is −4.77 to −2.15 (so it is not luck). OpsLens was lower on **14 of 14** of those days.

Two worlds. The dashboard has a switch at the top:

- **Food delivery (simulated)** — a support floor we generated. We know the truth there, so we can replay decisions and prove they help.
- **IT service desk (real, public)** — a real company's IT help desk, from a public dataset (UCI ServiceNow log, 24,918 incidents). It proves the ideas work on real data, not just on ours.

What makes it different (our "moats").

1. **Proof, not promises** — every claim is tested on unseen days, with fair comparisons and intervals.
2. **Trust** — every number clicks through to where it was computed; the AI never writes a number.
3. **Touch it** — judges can change things (What-if, Plant-a-problem, Ask OpsLens) and see the result live.

And one unusual thing: **we wrote down the rule for each experiment before running it**, and we show the ones we failed. That is the screen "Rules we wrote first".

2. Words you need

Word	Meaning in plain words
Ticket / complaint / incident	One customer problem. "Incident" is the IT-desk word.
SLA (deadline)	The promised time to fix it. Ours: urgent (P1) 1 hour, P2 4 hours, P3 24 hours.
Breach / missed deadline / missed promise	The ticket was fixed after its deadline. Our main score is the **% of tickets that breached**.
Agent	A support person. Our floor has 40 agents in 4 teams.
Team	A: live orders (late delivery). B: payments & refunds. C: order accuracy (missing / wrong item). D: account, app bugs, restaurant partners.
Skill	Which kinds of tickets an agent is trained for. Wrong skill = 1.6× slower and more re-opens.
Policy	A rule for who takes which ticket. OpsLens is one policy; the baselines are others.
Baseline	A normal way of working that we compare against (see Decision 5).
Industry standard	Our name for Baseline D: one shared queue, ordered by priority then deadline, given to any agent with the right skill. It is how helpdesk tools usually route. We even give it our AI classifier, so it is a strong opponent.
Simulator	A program that plays out a whole day minute by minute: tickets arrive, agents work, tickets finish late or on time.
Seed / luck	Random parts of the day (how long each job takes, who is absent). A seed fixes the luck so it can be repeated. We run 5 seeds and average.

Word	Meaning in plain words
Common random numbers	Every policy gets *exactly the same luck*: the same ticket takes the same time whoever handles it. So differences come from decisions, not luck.
Train / validation / test days	56 days in total. Days 1–35: learning. Days 36–42: choosing settings. Days 43–56: the final exam, **never used for any choice**.
Held-out	Not used for learning or choosing. "Held-out days" = the test days.
95% interval (CI)	A range that shows how sure we are. If the whole range is below zero, the improvement is real, not luck.
Pre-registration	Writing the rule that decides an experiment *before* running it, and committing it in git, so nobody can change the rule after seeing the result.
AUC	How well a model ranks: the chance a real "will be late" ticket gets a higher risk than an on-time one. 0.5 = coin toss, 1.0 = perfect.
Rescue	How much serving a ticket *now* lowers its chance of being late. OpsLens prioritises rescue, not just risk.
Bottleneck	Hours when the work coming in is more than the people on shift can handle.
Evidence card	One recurring problem, shown with a number, a comparison, and example tickets.
Playbook / fix at the source	An action outside the support floor that stops a problem at its origin (for example: fix the phone menu, cross-train agents).
Shadow mode	OpsLens only watches and warns; people still decide. How a real company would try it first.
LLM	A large language model (Gemini, Groq's Qwen). We use them for understanding words, never for numbers.

3. The problem statement, and how we answered each line

The PDF (DEV_103) asks for five things the system should do, four suggested technical parts, and four deliverables. Here is each line and our answer.

DEV_103 says	Our answer	Proof	Where on screen
Identify patterns in history	Evidence cards for 5 kinds of problems: capacity, skill gap, volume spike, partner issue, process issue	Found 5 of 5 planted problems; on the real desk, 3 of 3 findings still held on later months	Problems
Predict bottlenecks, delays, SLA breaches	48-hour forecast per team with an 80% band; each waiting ticket's chance to be late, with reasons	Risk ranking AUC 0.977; on real data 0.857 vs 0.552 for the desk's own priority field	Risks
Recommend actions (urgency, capacity, business impact)	Automatic dispatch (who takes what now), fixes at the source, staffing changes, *Your plan*	All 18 recommendations together: −48.8 missed deadlines over the test days	Actions, Your plan

DEV_103 says	Our answer	Proof	Where on screen
Explain decisions	Top reasons per ticket, evidence per problem, the runner-up agent for each assignment, every number clickable	Removing the top reason changes the risk more than a random one in 98.9% of cases	Risks, every number
Measure impact (turnaround, productivity, quality)	Simulator replays, results under the three headings, intervals	2.0% vs 5.5%; 14 of 14 days	Overview → Results, Proof
Tech: ML for forecasting + anomalies	LightGBM forecast + conformal band; robust z-score spike detection	80% band covered 79.2% of hours	Risks → Next 48 hours
Tech: NLP / LLMs	Local classifier + LLM for unsure tickets; LLM routes Ask questions; LLM writes words, not numbers	Real benchmark BANKING77: 91.0%	Proof → How we tested
Tech: optimisation	Min-cost flow assignment; CP-SAT roster tried	Honest: at our size a greedy rule ties it (section 7)	Proof → Rules
Tech: dashboard incl. performance tracking	Six screens, Live now, day-by-day tracking	Lower than the industry standard on 14 of 14 days	Overview
Deliverable 1: a public dataset	The real IT desk on every screen	Shadow mode: 53 of 102 real misses warned in time vs 16	IT service desk
Deliverable 2: dashboard of insights + bottlenecks	The dashboard	—	everywhere
Deliverable 3: explainable recommendations	Reasons on every recommendation and assignment	98.9% faithfulness	Risks, Actions
Deliverable 4: baseline vs proposed, measurable	4 baselines + published scheduling rules, intervals, 30 random worlds	Beat the industry standard in 29 of 30 worlds we did not design	Proof

4. The big decisions

For each big decision: **what we chose**, **why**, and **why not the other option**.

Decision 1 — One project, focused on support operations

- **Chose:** a decision engine for a *support floor* (customer complaints), with food delivery as the main story, and a general engine underneath.
- **Why:** the problem statement's own example is support tickets ("1,000 support tickets daily… SLAs"). Eternal runs food delivery; Hinglish complaints and lunch/dinner peaks make the story real to them.
- **Why not "general business data" (sales, workflows)?** Doing everything shallowly loses to doing one thing deeply and proving it. We say the engine works on any "work item" (something that arrives, has a type, a deadline, and a skilled handler) and show two kinds (food complaints, IT incidents).
- **Why not pivot fully to the public IT data?** We decided (owner, 18 Oct): "be good at general, insanely good at food". The food world is where we can *prove* decisions; the real desk is where we show it holds.

Decision 2 — Generated food data + one real public dataset

- **Chose:** generate the food-delivery floor ourselves (56 days, 71,530 tickets, 40 agents), and add the real UCI ServiceNow incident log (public, CC BY 4.0).
- **Why generate?** No company publishes support tickets with timestamps, agents, skills and deadlines. Without those you cannot test *decisions*. Generating also lets us **plant problems on purpose** and check whether OpsLens finds them.
- **Why the UCI log?** It is a real company's IT desk, with a real "made SLA" flag, real teams and hand-offs, and a clear licence. It answers "you only tested on your own data".
- **Why not the popular Kaggle "Zomato delivery" data?** We checked (18 Oct): it has no licence, and it is itself generated (delivery points shifted by exact steps, pickup times always 5/10/15 minutes). Using it would hurt our credibility, not help it.
- **Ticket text:** Bitext (a public customer-support text set) for the common categories, plus our own templates (dishes, restaurants, Hinglish) for food-specific ones. We say openly that text is partly synthetic, and we also test the classifier on a real text benchmark (BANKING77).

Decision 3 — A simulator, so decisions can be tested

- **Chose:** our own small discrete-event simulator (`opslens/simulate.py`).
- **Why a simulator at all?** A prediction can be scored on history, but a *decision* changes what happens next. The only fair way to compare "who takes which ticket" is to replay the same day under each policy.
- **Why our own and not a library (like SimPy)?** The owner must explain every line to Eternal's engineers. A few hundred lines with a heap of events is explainable; a framework is a black box.
- **How we trust it:** hand-calculated tiny cases must match exactly, and a big random run must match the classic Erlang-C formula from call centres (10.97 vs 10.79 minutes average wait, within 5%).
- **A fix we found (18 Oct):** at first, a transfer used up a new "luck draw", so a policy could re-roll a long job by misrouting it. We fixed it (the k-th piece of real work always uses draw k) and re-ran everything. Finding and fixing that kind of thing is why the numbers are believable.

Decision 4 — Fair comparison rules

- **Same customers, same luck (common random numbers), same agents, same hours** for every policy.
- **Choices only on validation days**; test days run once. When we changed anything later, we generated a **brand-new "sealed" world** after freezing the model and ran it once.
- **Why so strict?** A judge's first attack is "you tuned it to win". These rules make that impossible.

Decision 5 — The baselines (who we compare against)

Name	How it works	Missed (test days)
A — First come, first served	each team's oldest ticket first	13.1%
B — Urgent first	highest priority first, then oldest	6.8%
C — SLA due	the ticket closest to its deadline first (a common helpdesk view)	5.38%
D — Industry standard	one queue (priority, then deadline), any agent with the right skill; uses our AI classifier	**5.51%**

Name	How it works	Missed (test days)
Least slack first (published rule)	the ticket with the least time to spare, given its expected work	3.94%
Generalised c-mu (published rule)	value per minute of work	5.27%
OpsLens	rescue-based value + optimiser	**2.05%**

- **Why D is the lead comparison:** it is the strongest *normal* way of working, and we gave it our own AI triage, so we cannot be accused of beating a strawman.
- **Why published rules too:** a professor will ask "did you beat the best known rules, not just helpdesk defaults?" Yes, on our floor. (Where a published rule beats us — sustained overload — see section 7.)

Decision 6 — The OpsLens policy: value the *rescue*, not just the risk

- **Chose:** every few minutes, for each waiting ticket, OpsLens asks the risk model: "chance of being late if it waits" and "if served now". The difference is the **rescue**. Value = urgency × business impact (gold customers count 1.5×, order value counts a little) × (rescue + time already waited).
- **Why rescue and not just risk?** A ticket with 7 minutes left is almost certainly late whatever we do; serving it first can sink two others that could still be saved. Rescue puts effort where it changes the outcome. Proof: with a plain logistic regression *ranking* almost as well (AUC 0.957 vs 0.977), its rescue inside the dispatcher misses 3.87% vs 2.05%; with no risk model at all, OpsLens loses 2.15 points.
- **Safety rules:** a ticket far past its deadline still gets served (a "starvation guard", so no customer is abandoned); in a crisis (very long queue) the guard pauses to save what can still be saved.
- **History of the policy (each change tested on validation days first):**
- v1, v2 failed the +25% volume stress test (late tickets crowded out savable ones).
- v3 fixed that (a ticket past its deadline stops outranking savable ones).
- v4 (price cross-skill work in risk units) was **not adopted**: its gain was not clearly above zero.
- **v5 (shipped):** if no trained agent who speaks the customer's language will be on shift before the deadline, a rescue by another team pays no penalty. Confirmed on 20 fresh seeds; test days 2.21% → 2.05%.
- v6 (an "overload mode") is being tested now (section 13).

Decision 7 — The optimiser: min-cost flow (and why we are honest about it)

- **Chose:** at each decision, solve "which free agent takes which ticket" as a min-cost flow problem (Google OR-Tools), maximising the total value.
- **Why:** it is the textbook exact method for one-to-one assignment, fast (5,000 waiting tickets × 500 agents in a fraction of a second), and handles agents with several free slots.
- **The honest part:** decisions happen the moment anything changes, so usually only *one* agent is free; then the best global match equals each agent taking their best ticket. Our own test shows a greedy rule ties min-cost flow (−0.016 points), even with 3 chats per agent (+0.01). **The win comes from the value (prediction), not the matching.** We say that ourselves rather than wait for a judge to find it.

Decision 8 — LightGBM for risk and forecast (not deep learning)

- **Chose:** LightGBM (gradient-boosted trees) for the risk model and the forecast.
- **Why:** best-in-class on tabular data, runs on a laptop with no GPU, fast to retrain, and gives exact explanations (TreeSHAP) built in.
- **Why not deep learning?** Slower, needs more data, harder to explain, and the rules said free tools and no GPU. We still tested two modern "foundation models" as challengers (Chronos-2 for forecasting); they did not beat ours by our pre-set margin, so we kept LightGBM.
- **Calibration:** the risk model's probabilities are adjusted (isotonic calibration) on validation days so "70%" really means about 70%.

Decision 9 — Reading complaints: a cascade, local first

- **Chose:** a cheap local text model (TF-IDF + logistic regression) for every ticket; the LLM only for the few the local model is unsure about (about 1.5%).
- **Why:** fast, free and private; the LLM is used where it actually helps. On our text the local model is 98.0% accurate, the cascade 98.1%.
- **Why not send everything to the LLM?** On the real BANKING77 benchmark, the LLM alone scored 79.0%, below our local model (90.3%), and it is slower and costs quota. The cascade got 91.0%; with only 10 examples per category it lifted accuracy from 68.5% to 78.7%. Each tool where it is strong.
- **Privacy:** phone numbers, emails, order IDs and similar are masked before any text leaves the laptop (`opslens/privacy.py`).

Decision 10 — The LLM never writes a digit

- **Rule:** when an LLM writes text (the brief, cluster names, Ask answers), it can only refer to numbers by key, like `{{opslens_breach}}`. Our code checks the reply: any digit or unknown key → rejected → a fixed template is used instead. Our code fills in the real values.
- **Why:** LLMs invent numbers. A single made-up number on screen would destroy the trust story.
- **Ask OpsLens** follows the same idea: the LLM only *chooses a tool* (what-if, forecast, at-risk, …) and copies the user's own numbers; the engine computes the answer. It is a router, not a chatbot. If the LLM is offline, keyword rules answer (also Hinglish words like "kal", "kitne", "kaun").

Decision 11 — Forecast with a range, not a single line

- **Chose:** forecast arrivals per category per hour for 48 hours, with an 80% and 90% "conformal" band built from past errors.
- **Why a band?** Staffing needs "how bad could it be", not just "on average". The band is honest: the 80% band covered 79.2% of real hours. A bottleneck is when the top of the band exceeds what the team can do.
- **Score used:** WAPE (total error ÷ total volume): ours 0.305 vs 0.415 for "same hour last week".

Decision 12 — Staffing: Erlang C and budget-neutral moves

- **Chose:** from the forecast's upper band, compute how many agents each team needs per hour (Erlang C, the call-centre formula), then propose moves that keep **the same people and the same hours** (shift a start time, swap a day off).

- **Why budget-neutral?** "Hire more people" is not a decision a team lead can make today; moving the hours they already have is.
- **Honest result:** most single moves have no clear effect alone; we built an exact CP-SAT roster optimiser too, and it did not beat the simple moves on missed deadlines (section 7). We show that.

Decision 13 — Fixes at the source (playbooks)

- **Chose:** each problem card maps to an action for the team that can remove it: partner ops, product (phone menu), training (cross-training), customer comms (status banner, self-serve refunds). Each is replayed on the test days to measure its effect.
- **Why:** routing faster treats symptoms; the best ticket is the one that never needs to exist.
- **Best one:** cross-training two Hindi-speaking agents on payment failures and refunds: **26.8 fewer missed deadlines** over the test days, alone. Any effect size that is a business guess (for example how many customers a status banner stops from writing in) is labelled as an assumption with a range.

Decision 14 — Pre-registration and the "Rules we wrote first" screen

- **Chose:** before every serious experiment, write the success rule in `docs/PLAN_V4.md` (or the module's explain file) and commit it; git keeps the timestamp. Then run once and report whatever happens.
- **Why:** it is the strongest possible answer to "you only show what worked". Showing our misses is what makes the hits believable.
- **Why show the failures on screen?** Rigour was our best score but invisible. The ledger makes it the thing judges remember. (A planned improvement: group it into "core claims" vs "stretch bets", because a quick glance at "kept vs missed" counts can mislead — section 13.)

Decision 15 — Tests beyond our own world

- **Sealed worlds:** a new world generated *after* the model was frozen, run once. Latest: OpsLens 2.2% vs 5.8% for the industry standard (difference −3.64, interval −5.08 to −2.20).
- **30 random worlds:** volume, skills, deadlines, absences, team sizes, chats per agent all drawn at random from wide ranges fixed in advance. OpsLens beat the industry standard in **29 of 30**.
- **Agents with 2–3 chats at once:** OpsLens still wins in all four settings (−3.13 to −3.29 points).
- **The physical floor:** we proved, ticket by ticket, which deadlines could not be met even with unlimited agents (1.31% of tickets). OpsLens captures about 83% of the improvement that is physically possible. (The owner chose not to show this on screen; mention it only if asked.)

Decision 16 — The real desk: what we can and cannot claim

- **Can claim:** OpsLens warns early. One hour after an incident opens, OpsLens's risk ranks future misses far better than the desk's own priority field (AUC 0.857 vs 0.552). In **shadow mode** on the busiest real day (Thu 5 May 2016), OpsLens warned about 53 of the 102 real misses in time; the priority field 16. Its findings (waits on outsiders, slow pickup, a category bouncing between teams) **still held on later months**, 3 of 3.
- **Can claim with a stated assumption:** if the desk fast-tracked a fifth of new incidents (picked up at the 1-hour mark), OpsLens's choice would save 108 missed deadlines; the priority field's 29.

- **Cannot claim:** better *routing* than the real desk. A one-line rule ("fast-track whatever is still unassigned after 1 hour") saves 115 — a tie. And we cannot replay the real desk with our decisions: the log does not record how long people actually worked. We tried a calibrated "digital twin"; it failed its own validation (it missed 27.9% where reality missed 22.4%), so we show no twin result.
- **Why this matters:** judges trust a team that draws the line in the right place.

Decision 17 — Technology: plain and offline

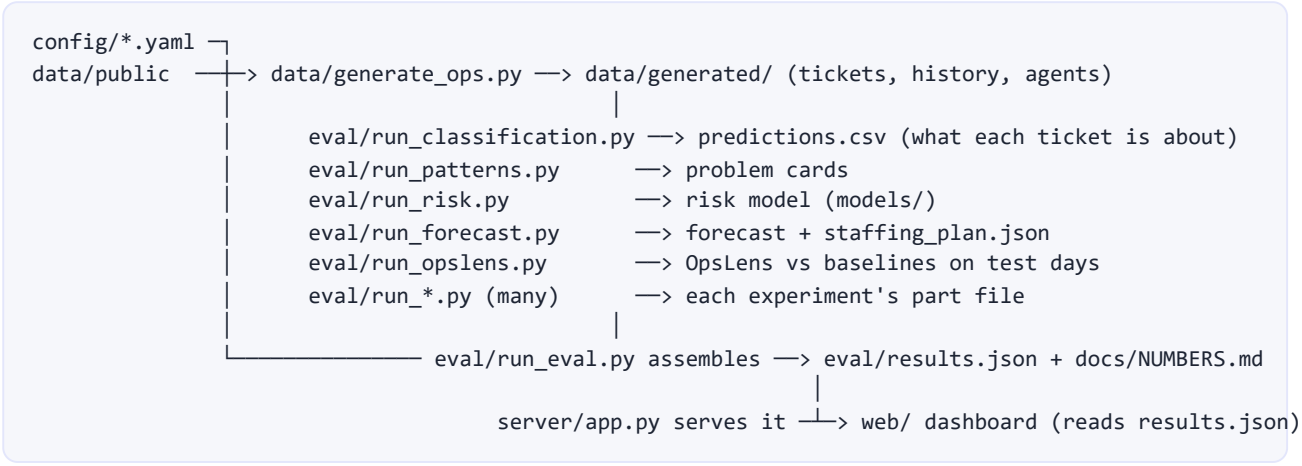
Chose	Why	Not chosen
Python 3.12, pandas, NumPy, scikit-learn, LightGBM, OR-Tools	free, fast, standard	—
FastAPI server	small, typed, easy to explain	Flask (fine too), Django (too big)
Plain HTML + JavaScript + CSS, Chart.js bundled	works with Wi-Fi off; every line readable	React (build step, harder to explain), Streamlit (looks generic, less control)
Free LLM tiers (Gemini Flash-Lite, Groq Qwen), with a cache	free; cached answers keep the demo offline	paid APIs
SQLite cache for LLM replies	no server to run	Redis etc.
Results in one JSON file	one source of truth, easy to check	a database

Decision 18 — The dashboard design choices

- **Six screens = six questions a manager asks:** what is happening? why does it go wrong? what will go wrong? what should we do? does it work? will it work on my data?
- **Two worlds** with one switch, same screens: simulated food floor and the real IT desk.
- **Accept / Override → Your plan:** the manager decides the big things (fixes, staffing); OpsLens decides the thousand small ones (who takes which ticket) and explains each. Accept shows the measured effect; Simulate replays a day with the plan; Override records why (a manager's "no" is information).
- **Live now:** today's weekday simulated on the laptop at the real clock — the demo is alive, not a slide.
- **Festive look, characters, splash:** the owner's request (24 Oct), built by Claude 2. It makes the demo memorable. Watch out: on a projector, the numbers must stay above the fold (section 13).

5. How the engine works, step by step

Think of it as a pipeline. Each step writes files the next step reads.



Step by step, in words:

1. **Generate the world** (`data/generate_ops.py`): 56 days of tickets with lunch and dinner peaks, weekend lift, two sale days; 40 agents with shifts and skills (`config/roster.yaml`); five planted problems (written to `data/planted.json` , which **only** `eval/` **may read** — the engine never peeks). The history is produced by running Baseline A, so it looks like a normal floor.
2. **Read the complaints** (`opslens/classify.py`): local model + LLM for unsure ones.
3. **Find patterns** (`opslens/patterns.py`): KPIs by every slice, where the time goes, spikes (robust z), each finding as an evidence card. `opslens/clusters.py` finds sub-issues inside a category and the LLM names them.
4. **Predict** (`opslens/features.py` , `opslens/risk.py` , `opslens/forecast.py`): risk uses only what is known at that moment (a test proves no future data leaks in); forecast with bands.
5. **Decide** (`opslens/optimize.py` , `opslens/policies.py` , `opslens/staffing.py` , `opslens/playbooks.py`).
6. **Prove** (`opslens/simulate.py` , `opslens/world.py` , `opslens/impact.py`): replay test days for every policy and seed; compute KPIs under turnaround / productivity / service quality; intervals.
7. **Assemble** (`eval/run_eval.py`): one file, `eval/results.json` , checked against `opslens/results_schema.py` ; `docs/NUMBERS.md` is written from it.
8. **Show** (`server/app.py` + `web/`): the dashboard reads only `results.json` , plus a few live endpoints (What-if, Plant, Your plan, Live now, Ask, Upload) that run the engine on demand.

6. What we can say — the results

Main result (food floor, 14 test days, 5 seeds, same customers and luck):

- Missed deadlines: OpsLens 2.05%, industry standard 5.51%, SLA due 5.38%, urgent first 6.8%, first come 13.1%. Difference vs the industry standard −3.46 points (95% interval −4.77 to −2.15).
- Day by day: lower on 14 of 14 days. Worst day for both: the sale day, Mon 19 Oct (3.98% vs 10.60%).
- Your plan: all 18 recommendations together −48.8 missed deadlines (interval −63.1 to −34.5).

Beyond our world:

- Sealed world made after the freeze: 2.2% vs 5.8%.
- 30 random worlds: beats the industry standard in 29 of 30.
- Agents with 2–3 chats: still wins in all four settings.

Prediction:

- Risk model AUC 0.977 (logistic regression 0.957, a "time left" rule 0.884). Calibrated (Brier 0.055).
- Forecast WAPE 0.305 vs 0.415 for same-hour-last-week; 80% band covered 79.2%.
- Bottleneck alerts: real desk precision 77.6%, recall 92.8%.

Explanations: faithful in 98.9% of cases; for the planted Hindi problem the reasons name language or skill 98.6% of the time (63% for other tickets of the same team).

Language / AI: BANKING77 91.0% with the cascade; Ask OpsLens routing 93.3% vs 61.7% for keyword rules, Hinglish 18 of 18.

Real desk: early warning AUC 0.857 vs 0.552; shadow mode 53 vs 16 of 102 real misses; findings 3 of 3 hold later; fast-track estimate 108 saved vs 29 for the priority field (115 for the "still unassigned" rule: a tie).

7. What failed, and what we learned

We keep these on screen. Each was a rule written before the run.

Experiment	What we hoped	What happened	Lesson
Policy v4	Pricing cross-skill work better would help	−0.08 points, not clearly above zero → not adopted	Small ideas need big evidence
30 random worlds	OpsLens best of all six methods in 27 of 30	19 of 30 (but 29 of 30 vs the industry standard)	All losses are floors in sustained overload; there least slack wins the count, OpsLens protects gold and late customers
Real-desk fast-track	OpsLens's pick beats every other picker	Ties a one-line "still unassigned" rule (115 vs 108)	Risk is not rescue; fast-tracking only helps tickets still waiting
Rescue picker (second look)	Beats the simple rule	160 vs 115, but the interval touches zero → not claimed	Close is not proven
Detector v2 (real log)	Find more planted problems	No setting met the false-alarm limit → kept v1 (11 of 60)	Real data is noisy; spikes are easy, slowdowns are hard
LLM root-cause naming	LLM names causes better than rules	61 vs 62 of 62 → ties	Use the LLM for words, not for judgement
CP-SAT roster	An exact optimiser beats greedy staffing	+0.21 worse on test days → not adopted	The planning score (Erlang C) was the limit, not the search
Real-desk digital twin	Replay the real desk with our decisions	Failed its validation gate (27.9% vs 22.4%) → not shown	A log without work times cannot be replayed honestly

Experiment	What we hoped	What happened	Lesson
Min-cost flow vs greedy	The optimiser adds a lot	Ties greedy	The prediction (rescue value) is the win

The big lessons:

1. The value of OpsLens is in **predicting rescue**, not in clever matching.
2. Under **sustained overload**, a different rule is better on raw counts; switching strategy needs a reliable trigger (our first switch fired too often on normal days).
3. On **real data**, OpsLens is strongest at **early warning**, not routing.
4. Writing the rule first protected us from fooling ourselves many times.

8. The dashboard, screen by screen

Top bar: the **world switch** (Food delivery / IT service desk), **Ask OpsLens**, **Neon/Light** theme, **Your plan**, **Story mode**. Left side: the six screens, and a small live panel.

Screen	Food world shows	Real desk shows	Main files
Overview	*Live now* (today simulated at your clock) and *Results on the test days*	Overview (fast-track outcome) and *Shadow mode* (a real day replayed)	`tabs/live.js`, `tabs/overview.js`, `tabs/realdesk.js`, `tabs/shadow.js`
Problems	Evidence cards, where time goes, clusters, *Plant a problem*	Real findings, "still true on later months"	`tabs/patterns.js`, `tabs/realdesk.js`
Risks	*Customers about to miss* (reasons, what would help) and *Next 48 hours*	Early warning and week-ahead alerts	`tabs/atrisk.js`, `tabs/forecast.js`
Actions	Fixes at the source, staffing, automatic dispatch with reasons; Accept/Override	Real-desk actions	`tabs/recommendations.js`, `plan.js`
Proof	*Side by side*, *Beyond our world*, *Try a what-if*, *Rules we wrote first*, *How we tested*	*Against the real desk*, *Rules*	`tabs/compare.js`, `tabs/worlds.js`, `tabs/whatif.js`, `tabs/ledger.js`, `tabs/tested.js`
Your data	Upload any CSV and map its columns (UCI example ready)	same	`tabs/upload.js`

Every number is a button: click it to see where it comes from (`web/js/evidence.js`). **Story mode** walks the nine steps of the demo (`web/js/story.js`), in the same order as `docs/DEMO_SCRIPT.md` .

9. The repository, folder by folder and file by file

Top level

File	What it is
`README.md`	The front page: the problem statement answered line by line, what we don't claim, how to try it
`CLAUDE.md`	The plan and the rules for the AI assistants (and for us): hard rules, module list, milestones
`PROGRESS.md`	The log: what was built when, the numbers, open issues
`Makefile`	Short commands: `setup`, `data`, `ping`, `gen`, `test`, `eval`, `serve`, `demo` (on Windows: `mingw32-make <target>`)
`requirements.txt` / `requirements.lock.txt`	Python packages (and exact versions)
`pytest.ini`	Test settings
`.env` (not in git)	Your API keys. Never commit it. `.env.example` shows the format.

`config/` — every setting in one place

File	What it sets
`calendar.yaml`	Start date (31 Aug 2026), 56 days, the train/validation/test split, sale days
`generator.yaml`	How the food floor is generated: volumes, peaks, categories, languages, handle times, planted problems. **The engine never reads this** (only the generator and simulator do)
`roster.yaml`	All 40 agents: team, skills, languages, shift start, days off (fictional names)
`sla.yaml`	Deadlines: P1 60 min, P2 240 min, P3 1440 min
`optimizer.yaml`	OpsLens's weights (chosen on validation days, frozen)
`playbooks.yaml`	Business assumptions for the fixes at the source (labelled, with ranges)
`providers.yaml`	Which LLMs to use (Gemini, Groq, a mock for tests), limits and models

`opslens/` — the engine (each file has a long explanation at its top)

File	What it does
`config.py`	Loads the YAML files and `.env`
`privacy.py`	Masks personal data before text goes to any LLM
`ingest.py`	Loads and checks a ticket export; computes waits, resolution times, breaches
`kpis.py`	KPI tables by any slice (team, category, language, hour…)
`classify.py`	The classifier cascade (local model + LLM when unsure)
`clusters.py`	Sub-issues inside a category; the LLM names them

File	What it does
`patterns.py`	Recurring problems + spike detection → evidence cards
`features.py`	Risk-model inputs built only from what is known at that moment
`risk.py`	The risk model: chance to be late, reasons, "what would help"
`forecast.py`	48-hour forecast with bands
`staffing.py`	Erlang-C needs and budget-neutral moves
`roster_opt.py`	The exact CP-SAT roster (tested, not adopted)
`optimize.py`	Ticket value and the min-cost-flow assignment; the runner-up text
`policies.py`	All policies: baselines A–D, published rules, OpsLens (and its overload-mode experiment)
`simulate.py`	The discrete-event simulator
`world.py`	Builds one simulated world (tickets + roster + seed + luck)
`impact.py`	KPIs under turnaround / productivity / service quality, intervals over seeds
`floor.py`	The physical floor proof (unavoidable misses)
`whatif.py`	Re-runs one day with changes (What-if panel)
`plan.py`	*Your plan*: applies accepted items and measures them
`playbooks.py`	Fixes at the source as scenarios to replay
`brief.py`	The morning decision brief (LLM words, code numbers, template fallback)
`ask.py`	Ask OpsLens: question → tool → answer, numbers from code
`live.py`	Live now: today's day simulated and served minute by minute
`realdesk.py`	The real desk: cards, early warning, forecast, routing check
`realdesk_fasttrack.py`	The fast-track benchmark on the real desk
`realdesk_watch.py`	Spike / slowdown / bounce detectors on the real log
`realdesk_shadow.py`	Shadow-mode data for real days
`realdesk_twin.py`	The digital-twin attempt (failed its gate, kept for honesty)
`results_schema.py`	The contract for `results.json` (what fields exist)
`llm/`	The LLM layer: `client.py` (one entry point), `openai_compat.py` (Gemini/Groq), `mock.py` (tests), `cache.py` (SQLite), `ratelimit.py` (free-tier limits), `base.py` (shared types)
`adapters/uci_incidents.py`	Turns the UCI event log into one row per incident, plus "as of" features

`data/`

File	What it is
`generate_ops.py`	Generates the 56-day food floor and the history

File	What it is
`text_bank.py`	Ticket text (Bitext + our templates, Hinglish)
`make_llm_templates.py` / `llm_templates.json`	One-time extra message variety from an LLM (committed)
`plant.py`	Random planted problems (blind tests, Plant-a-problem)
`planted.json`	The truth about planted problems — **only `eval/` reads it**
`SOURCES.md`	Every dataset, link and licence
`public/`	Downloaded datasets (Bitext, UCI, BANKING77)
`generated/`	The generated world (not in git): tickets, history, predictions, models, staffing plan

`eval/` — every experiment (the source of every claim)

File(s)	What it measures
`run_eval.py`	Runs the steps and assembles `results.json` + `docs/NUMBERS.md` (`--assemble` = only rebuild)
`results.json`	**The single source of every number** (about 3 MB)
`fixture_results.json` , `make_fixture.py` , `results.schema.json`	Fake results for UI work (marked FIXTURE) and the schema
`parts/`	Each experiment's own result file, assembled into `results.json`
`run_classification.py` , `run_patterns.py` , `recall.py` , `run_blind.py`	Reading complaints, finding problems, planted and blind recall
`run_risk.py` , `run_forecast.py`	Prediction quality and staffing plans
`run_baselines.py` , `run_opslens.py` , `run_baseline_c.py` , `run_best_baselines.py`	The main comparisons
`run_ablations.py`	What each part of OpsLens is worth; stress tests
`run_design_v2..v6.py` , `run_confirm_v5.py`	Policy changes, designed on validation days and confirmed
`run_floor.py`	The physical floor
`run_sealed_world.py`	Brand-new worlds run once
`run_worlds.py` , `run_world_compare.py`	The 30 random worlds
`run_concurrency.py`	Agents with 2–3 chats
`run_tracking.py`	Day by day, team by team
`run_playbooks.py` , `run_plan_items.py`	Fixes at the source and *Your plan* items
`run_realdesk.py` , `realdesk_plants.py` , `run_realdesk_twin.py`	The real desk, its blind plants, the twin
`run_nlp_bench.py` , `run_llm_eval.py` , `llm_eval/`	BANKING77, LLM root causes, Ask routing (60 questions written first)

File(s)	What it measures
`run_w7.py` , `run_w8.py`	Strong baselines for prediction; explanation faithfulness
`run_roster_opt.py`	CP-SAT roster
`challengers/`	Optional: foundation-model challengers (separate requirements)

`server/`

File	What it does
`app.py`	The web server: serves the dashboard and `results.json` ; live endpoints What-if, Your plan, Live now (warmed at start), Ask, Upload
`demo.py`	`make demo` : starts offline, refuses fake (fixture) results
`plant_demo.py`	Live Plant-a-problem
`upload.py`	Analysis of an uploaded CSV

`web/` — the dashboard (plain HTML/JS/CSS)

File	What it does
`index.html` , `app.js`	The page and its start-up
`styles.css` , `theme.css` , `neon.css`	Looks: base, layout, the neon theme (Light is a switch)
`js/data.js` , `js/evidence.js` , `js/format.js`	Read numbers by path; click-to-evidence; number formats and definitions
`js/router.js` , `js/ui.js` , `js/charts.js` , `js/motion.js`	Screens and tabs; building blocks; charts; count-up numbers
`js/plan.js`	*Your plan* panel (Accept/Override/Simulate)
`js/ask.js`	The Ask OpsLens bar
`js/story.js` , `js/storycards.js`	Story mode; "Out of 1,000 customers" and "Follow one ticket"
`js/replay.js` , `js/floormap.js` , `js/intro.js` , `js/mission.js`	Replays, the floor picture, the cold open, mission control band
`js/liveticker.js`	The small live panel in the side menu
`js/splash.js` , `js/fest.js` , `js/season.js` , `js/logo.js` , `js/mascot.js` , `js/characters.js` , `js/scenes.js` , `js/chase.js`	The festive splash and banner, seasonal logo, the OpsLens mark, "Lenny" and other characters, cartoon scenes
`js/themeswitch.js`	Neon / Light
`js/tabs/*.js`	One file per screen (see section 8)
`vendor/`	Chart.js and fonts, bundled so it works offline

`tests/` — 217 tests

They check, among other things: the simulator on hand-calculated cases and against Erlang C; that no future data leaks into risk features; privacy masking; the LLM never putting a digit on screen; every pre-registered helper; and `test_doc_numbers.py`, which checks the decimal numbers in the README, demo script, Q&A, cheat sheet and this guide against `results.json`.

`docs/`

File	What it is
`GUIDE.md`	This guide
`OWNER_CHEATSHEET.md`	One page to learn by heart before judging
`DEMO_SCRIPT.md`	The 7-minute demo, following Story mode
`JUDGE_QA.md`	80+ hard questions with short answers
`NUMBERS.md`	Every number, generated from `results.json`
`PRODUCTION.md`	How a real company would deploy it (shadow mode first), limits
`PLAN.md`, `PLAN_V3.md`, `PLAN_V4.md`, `CLAUDE_v1.md`	The plans over time and every pre-registered rule
`OPSLENS_IN_PLAIN_WORDS.md`	The earlier short explainer
`explain/*.md`	One plain-words explanation per module, each with a worked example
`collab/`	Notes between the two Claude sessions

`scripts/`

`download_data.py` (Bitext, UCI), `download_banking77.py`, `ping_llm.py` (checks your LLM keys work).

`cache/`

Saved LLM replies (SQLite), so the demo works offline and costs nothing twice.

10. How to run everything

```
python -m venv .venv
.venv/Scripts/python.exe -m pip install -r requirements.txt
copy .env.example .env        # optional free Gemini / Groq keys
mingw32-make data             # download the public datasets
mingw32-make gen              # generate the food floor (a few minutes)
mingw32-make test             # all tests
mingw32-make eval             # every experiment → results.json (long)
mingw32-make demo             # the dashboard, offline: http://localhost:8000/
```

Useful single commands:

- Rebuild `results.json` from the part files without re-running anything: `.venv/Scripts/python.exe eval/run_eval.py --assemble --keep-brief`
- Run one experiment: `.venv/Scripts/python.exe eval/run_<name>.py`
- Start the server for development: `mingw32-make serve`

Before a demo: start the server **5 minutes early** (Live now warms up), skip the splash with Esc, check Neon vs Light on the projector.

11. How we worked

- **Two people:** Jijnasu (owner, all technical decisions, presents the tech) and Sneha (story, demo, UI taste).
- **Two AI coding assistants:** "Claude 1" (Jijnasu's, the engine, reviews and merges) and "Claude 2" (Sneha's, reviews, UI, several experiments). They talk through **GitHub issues and pull requests** on the private repo `ChunChun451/opslens`.
- **Rules they work under** (in `CLAUDE.md`): free tools only; every number from `results.json`; the LLM never writes a digit; no tuning on test days; fair comparisons; the engine never reads the planted truth; privacy masking; every module explained in plain words; one milestone at a time; tests pass before merge.
- **Every merge is reviewed:** compare every value in `results.json` before and after (no existing number may move unless on purpose), run all tests, check the rule was written before the run.
- **Why say this openly:** judges will see the AI in the commit history. The honest framing is a strength: AI agents worked under rules that make their work checkable, and the team made every decision and can explain every part. (How to present this is the team's decision.)

12. The timeline

Date	What happened
15 Oct	Plan v2; repo, LLM layer, privacy, datasets (M1); generator, simulator, baselines (M2)
16–17 Oct	Classifier, patterns, risk, forecast, optimiser, OpsLens policy (M3–M7); fixes at the source
17–18 Oct	Claude 2 joins: review, baselines C and D, Ask OpsLens, cold open, sealed world
18 Oct	Simulator fairness fix; floor proof; v4 not adopted; **v5 shipped (2.05%)**; new sealed world
19 Oct	Plan v3: two worlds, six screens, *Your plan* with real Accept/Override, the real desk
19 Oct	Plan v4 (aim: 9–10 on every judging criterion); chats test passed; 30 worlds (19 of 30)
22 Oct	Claude 2: BANKING77, strong baselines, faithfulness, CP-SAT (not adopted), ledger UI
23 Oct	Real-desk second looks (both missed); diagnosis of overload losses (gold and late protected)
24 Oct	Live now and neon UI; every proof on screen; shadow mode; twin (failed gate); v6 rounds; fresh-judge review: tracking (14 of 14 days), Ask forecasts, decision brief, README, demo script, cheat sheet, this guide

Date	What happened
26 Oct (eve)	Freeze
27 Oct	Rehearsal, backup video
29–31 Oct	Finale

13. What is still open

- **v6 overload mode:** round 1 chose "value per minute of work in overload" (no worse than v5 on fresh seeds); its final test on 30 new worlds is running. Expected: it will not reach the 27-of-30 bar, and will go on the ledger as missed, with its numbers.
- **Screen fixes (Claude 2, issue #19):** a one-line headline on the first screen, festive elements below the fold; the day-by-day chart; the decision brief on Overview; Ask suggestions; deep links; fewer numbers per screen; and **grouping the ledger into core claims (all kept) and stretch bets**.
- **The team's AI story:** decide together how to say it, then say it first.
- **Rehearsal:** the cheat sheet and two full runs of the demo script; record the backup video.

14. If you only remember ten things

1. OpsLens finds **why** deadlines are missed, predicts **who** will miss next, decides **what to do**, and **proves** it on days it never saw.
2. **2.0% vs 5.5%** missed against the industry standard; lower on **14 of 14** days.
3. Same customers, same luck, same agents — choices on validation days only, test days run once.
4. It wins because it values the **rescue** (how much serving now helps), not just the risk.
5. The optimiser itself ties a greedy rule — the **prediction** is the win. Say it first.
6. Beyond our world: **29 of 30** random worlds; a sealed world made after the freeze; agents with 2–3 chats.
7. Real public data: early warning **0.857 vs 0.552**; shadow mode **53 vs 16** real misses caught in time.
8. Where it loses: **sustained overload** (least slack wins the count; OpsLens protects gold and late customers); real-desk **routing** (a simple rule ties). We show both.
9. The AI never writes a number; every number clicks through to its source.
10. We wrote every rule **before** the run, and the misses are on screen. That is why the hits are believable.